

Day 3

메모리 — 주소는 먼저 , 페이지는 나중에

2026.10.07 (수) · 8 교시 · 운영체제 수업 VM · 측정 단위 : page fault

```
$ ./05_cow 10
자식이 10 % ( 1280 페이지 ) 씬 → minflt +1280
$ ./05_cow 100
자식이 100 % (12800 페이지 ) 씬 → minflt +12800
```

가상 메모리를 page fault 수로 증명한다

1 교시

주소 공간

01_layout

2 교시

malloc
내부

02_malloc_path

3 교시

demand
paging

03_demand

4 교시

mmap

04_mmap_upper

5 교시

Copy-on-
Write

05_cow

6 교시

mlock ·
OOM

06 · 07

7 교시

오류 도구

08_bugs

8 교시

memlab

종합

측정 도구 3 가지

VmSize · VmRSS (/proc/self/status) · minflt · majflt (getrusage) · strace · valgrind · ASan

변수는 어디에 사는가

높은 주소 ▲

stack	지역 변수 ↓
mmap	큰 malloc · 라이브러리
heap	작은 malloc ↑
bss	초기값 없는 전역
data	초기값 있는 전역
rodata	"hello"
text	main()

낮은 주소 ▼

```
text    main()          0x558e243e636a
data    g_init          0x558e243e9010
bss     g_zero          0x558e243e9024
heap    malloc(64)      0x558e621062a0
mmap    malloc(1MB)     0x7fdbe08f1010
stack   local           0x7ffef820f044
```

```
$ setarch -R ./01_layout | head -1
text    main()          0x55555555536a
```

▶ 애니메이션 [D3-1 주소 공간 지도](#)

작으면 heap(brk), 크면 mmap

	heap (brk)	mmap
크기	128KB 미만	128KB 이상
주소	0x55... [heap]	0x7f...
free	glibc 가 보관 · 재사용	munmap 즉시 반환
시스템 콜	가끔 brk	매번 mmap/munmap

```
$ strace -e trace=brk,mmap,munmap ./02_malloc_path 256
mmap(NULL, 266240, PROT_READ|PROT_WRITE,
      MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fcd6e6e7000
munmap(0x7fcd6e6e7000, 266240) = 0
```

malloc 은 시스템 콜이 아니다

glibc 함수가 미리 받아 둔 메모리에서 잘라 주고,
모자랄 때만 brk · mmap 으로 커널에 간다

▶ 애니메이션 [D3-2 malloc 의 두 길](#)

주소는 먼저 , 페이지는 처음 만질 때

단계	VmSize	VmRSS	minflt
시작	2,692 kB	1,412 kB	+0
malloc(100MB)	105,096 kB	1,736 kB	+12
앞 50MB 페이지마다 1 바이트	105,096 kB	52,932 kB	+12,811
memset 100MB	105,096 kB	104,136 kB	+25,612

CPU 접근

>

MMU 확인

>

page fault

>

커널 연결

>

다시 실행

▶ 애니메이션 [D3-3 demand paging](#)

파일을 배열처럼 — page cache 를 직접

플래그	쓰면	용도
MAP_SHARED	파일과 다른 프로세스에 반영	파일 수정 · 공유
MAP_PRIVATE	내 복사본 (COW) · 파일 그대로	실행 파일 · 라이브러리
MAP_ANONYMOUS	파일 없이 0 으로 시작	큰 malloc

```
$ ./04_mmap_upper test.txt private ; cat test.txt
hello mmap world
$ ./04_mmap_upper test.txt ; cat test.txt
HELLO MMAP WORLD
```

온디바이스 AI

모델 가중치를 mmap → 실제로 읽는
부분만 page fault 로 RAM 에, 여러
프로세스가 같은 page cache 공유

▶ 애니메이션 [D3-4 mmap 과 page cache](#)

fork 는 복사하지 않는다 — 쓸 때만

자식이 0 % 씀

+0

minflt

자식이 10 % 씀

+1, 280

minflt

자식이 50 % 씀

+6, 400

minflt

자식이 100 % 씀

+12, 800

minflt

50MB = 12,800 페이지 → fault 수 = 쓴 페이지 수 (05_cow 실측)

```
pid_t pid = fork();
if (pid == 0) {
    for (long i = 0; i < limit; i += page) buf[i] = 2;    /* 이 페이지만 복사 */
    printf("minflt %+ld\n", minflt() - f0);
}
```

▶ 애니메이션 D3-5 Copy-on-Write

fault 를 미리 당기고 , 한도 안에서 죽인다

	malloc 중 fault	루프 중 fault	루프 시간
잠금 없음	+3	+8,194	13.04 ms
mlockall	+8,226	+1	0.12 ms

```
$ sudo systemd-run --scope -p MemoryMax=100M \
  -p MemorySwapMax=0 ./07_oom
90 MB 사용, VmRSS 93800 kB
Killed
$ sudo dmesg | tail -1
Memory cgroup out of memory: Killed process 2731 (07_oom)
```

NULL 검사로는 못 막는다

overcommit 0 에서 malloc 은 거의 항상 성공 .
부족은 memset(실제 사용) 순간 SIGKILL 로 온다 .

overcommit 2 (엄격) → 한도를 넘는 malloc 이 NULL

▶ 애니메이션 D3-6 mlock · OOM

바로 죽지 않는 오류를 도구로

오류	도구 없이	valgrind	ASan
leak	정상 종료 (0)	definitely lost 100 B	LeakSanitizer
use-after-free	쓰레기 값 · 0	Invalid read	heap-use-after-free
overflow	정상 출력 · 0	Invalid write	heap-buffer-overflow
double free	Aborted (134)	Invalid free()	attempting double-free

```
$ valgrind --leak-check=full ./08_bugs leak
$ make asan && ./08_bugs_asan overflow
```

valgrind: 재빌드 불필요 · 10~50 배 느림
 ASan: -fsanitize=address · 2 배 · 즉시 중단

▶ 애니메이션 [D3-7 메모리 오류 탐지](#)

네 가지 실험을 CSV 한 줄로

```
exp,size_mb,minflt,majflt,rss_kb,ms  
demand,64,16393,0,67012,25.03  
cow,64,16403,0,66352,52.81  
mmap,64,556,1,67212,11.31  
lock,64,1,0,68208,0.50
```

▶ 애니메이션 [D3-8 memlab 대시보드](#)

온디바이스 AI 메모리 원칙

1. 모델 가중치 → mmap 으로 필요한 부분만
2. 실시간 태스크 → SCHED_FIFO + mlockall
3. 장시간 동작 → valgrind · ASan 으로 누수 점검
4. 메모리 실험은 cgroup 한도 안에서

다음 시간 · Day 4 (10/8 목)

파일시스템 · sysfs

파일 I/O 의 비용 · page cache · inotify · 커널 모듈로 sysfs 속성 만들기

개념 워크시트 80 문항

Linux-day3/worksheet.html

애니메이션 8 종

Linux-day3/animation.html